



INDIA.RUNS

Build what next India runs on

Team Name : <<TEAM NAME>>

Team Leader Name : <<TEAM LEADER NAME>>

Problem Statement : Intelligent candidate discovery & ranking — rank 100K profiles for a Senior AI Engineer JD the way a great recruiter would, not by keyword matching.

Solution Overview

- **What is your proposed solution?**

- A hybrid ranking engine: deterministic profile-consistency checks → corroboration-gated structured JD-fit features → semantic (MiniLM bi-encoder, 5 JD facets) + BM25 fusion → behavioral availability multiplier → top-100 with fact-grounded reasoning per candidate.
- All heavy computation is precomputed offline and shipped as artifacts; the timed ranking step runs in ~2 s on CPU against a 300 s budget.

- **What differentiates it from traditional candidate matching systems?**

- Keywords are treated as claims, not evidence. A listed skill scores only when the title or career history corroborates it — measured on this pool, $P(\text{PyTorch} \mid \text{ML-titled}) = 21.6\%$ vs 1.2% for everyone else — so keyword stuffers get exactly zero credit.
- What people actually built (career narratives), where they built it (product vs consulting vs AI-native companies) and how they engage (recruiter signals) drive the rank; impossible profiles are detected and excluded; every score is explainable with no LLM at runtime.

JD Understanding & Candidate Evaluation

- **Key requirements extracted from the JD**

- Core: built retrieval / ranking / recommendation systems in production; evaluates ranking quality rigorously (NDCG, A/B); 5–9 yrs treated as a soft band; product-company shipping culture; LLM fine-tuning a plus.
- Logistics: Noida/Pune or willing to relocate (no visa sponsorship); notice ≤ 30 days buyable. Disqualifiers are graded like the JD's own language: consulting-only and research-only careers = hard kills ('will not move forward'); job-hopping, CV/speech-only, LangChain-only = heavy penalties ('probably not').

- **Which signals matter most / fit beyond keyword matching?**

- Career-history descriptions are the primary evidence, then title archetypes and company taxonomy; redrob behavioral signals — last-active, recruiter response rate, open-to-work, notice period, relocation — modulate the skill-match score multiplicatively, exactly as the signals doc prescribes.

Ranking Methodology

- **How does the system retrieve, score and rank?**
 - No hard retrieval gate — all 100K candidates receive all scores (a lexical gate would silently drop plain-language strong profiles). Semantic: MiniLM-L6-v2 embeddings vs 5 authored JD facets, mean of top-2 similarities. Lexical: BM25 vs a JD-derived query. Structured: ~20 features (title archetype, retrieval evidence, production & evaluation signals, corroborated skills, product ratio, experience band).
- **Models, algorithms, heuristics — and why**
 - Bi-encoder (22M params, CPU-friendly), BM25, and hand-weighted linear scoring. Interpretable by design: no labelled ground truth exists, so a trained learning-to-rank model would only learn our assumptions back, with less transparency.
- **How are signals combined into the final ranking?**
 - z-normalise $\rightarrow$ sigmoid $\rightarrow$ fuse $0.45 \cdot \text{structured} + 0.40 \cdot \text{semantic} + 0.15 \cdot \text{BM25}$; multiplicative kills (honeypot $\times 0.02$, consulting-only / research-only $\times 0.05$, non-tech title $\times 0.10$); $\times$ availability — a weighted geometric mean of recency, response rate, open-to-work, notice, location, interview completion (floor 0.25, so behavioral twins actually separate). Deterministic tie-break; every date anchored to the dataset's frozen clock, 2026-06-01.

Explainability & Data Validation

- **How are ranking decisions explained?**

- Programmatic generation from extracted facts only: real company names, matched retrieval terms, corroborated skills, signal values. Concerns are disclosed systematically — notice >30 days, dormancy, not open-to-work, overseas without relocation — and tone tracks rank.

- **How do you prevent hallucinations or unsupported justifications?**

- No LLM at runtime. Every clause renders from verified profile fields, so an unsupported claim is structurally impossible — and the output is deterministic, reproducing byte-for-byte across machines.

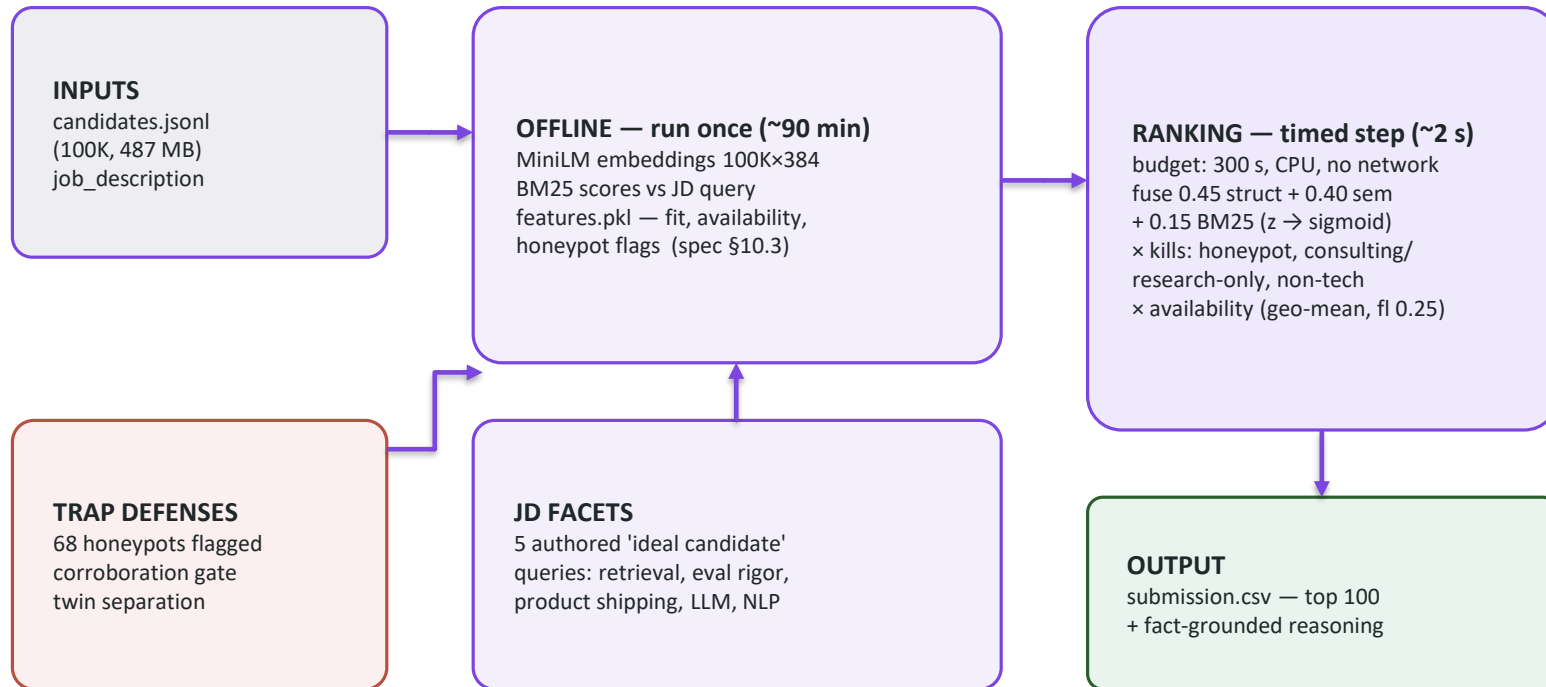
- **How are inconsistent, low-quality or suspicious profiles handled?**

- 68 planted honeypots caught by three record-internal checks: 'expert' skill with 0 months used; current job implying today ≠ 2026-06; career span contradicting stated experience by >3 yrs (with the ±1-month stint tolerance the data requires — 64,649 stints carry ordinary calendar rounding).
- Keyword stuffers die at the corroboration gate; behavioral twins are separated by the availability multiplier; the pre-submission audit enforces zero honeypots in the top 100.

End-to-End Workflow

- 1. JD analysis — requirements, graded disqualifiers, and 5 semantic facet queries authored from the role description
- 2. Offline precompute (~90 min, permitted by spec §10.3) — MiniLM embeddings for all 100K profiles, BM25 scores, and a structured-feature / availability / honeypot cache
- 3. Timed ranking step (~2 s of the 300 s budget) — load artifacts, fuse scores, apply kills and penalties, multiply availability, take the top 100
- 4. Reasoning — fact-grounded explanation per candidate, concerns included
- 5. Validation & audit — official validator, zero-honeypot check, twin-ordering check, manual top-50 review → submission.csv

System Architecture



Results & Performance

- **What results demonstrate ranking quality?**

- Top-100 is what a recruiter would expect for this JD: search / recsys / ML engineers, 4–9 yrs, at AI-native startups and product companies — surfaced from a pool that is 64% non-tech and only ~1% ML-titled.
- 0 of 68 detectable honeypots in the top 100 (DQ threshold: >10); behavioral-twin ordering verified; every risky row carries an honest, data-backed concern; facts in reasonings spot-checked against raw profiles with zero errors.

- **How are the runtime and compute constraints met?**

- Timed ranking step: ~2 s wall-clock vs the 300 s budget, CPU-only, no network, well under 16 GB. Offline precompute (~90 min) ships as artifacts per spec §10.3.
- All recency math anchors to the dataset's frozen clock (2026-06-01), never the wall clock — so reproduction is byte-identical on any machine, any day.

Technologies Used

- Python 3.11 + NumPy — vectorised scoring; 100K×384 similarity is a single matrix multiply
- sentence-transformers (MiniLM-L6-v2, 22M params) — the best quality-per-CPU-second bi-encoder at this scale; embeds the full pool offline without a GPU
- rank-bm25 — document-length-aware lexical signal fused alongside semantics
- Python stdlib for feature extraction and reasoning generation — deterministic, no-network, zero-hallucination runtime
- NPY / pickle artifact cache — moves every heavy step out of the timed window (127–311 s of JSONL parsing → ~2 s)
- Deliberately no GPU, no external APIs, no LLM at inference — the constraints reward small, sharp and reproducible over large and fragile

Submission Assets

- GitHub repository: <<GITHUB REPO URL>> — full source, README with one-command reproduction, precompute scripts, 5 commits of real iteration history
- Ranked output: submission.csv — top-100, passes the official validator
- Sandbox: <<SANDBOX LINK>> — hosted environment running the ranker end-to-end on a sample
- Demo video: <<DEMO VIDEO LINK>>
- submission_metadata.yaml at repo root mirrors the portal metadata



INDIA.RUNS

Build what next India runs on

THANK YOU